MULTICOMPRA · NODO

Fase 2 MULTICOMPRA en SpAdconApp

2 nuevos reportes para cerrar el ciclo OP → OC → Compra

Para: Carlos Cardozo

De: Emiliano Vitale (NODO)

Fecha: 28 de abril de 2026

Continuación de: `api-spadconapp-multicompra.pdf`

1) Contexto rápido

En la fase 1 ya implementaste 6 reportes `MULTICOMPRA_*` que andan perfecto. La PWA ya levanta locales, productos, stock, proveedores y autentica usuarios. Ahora tenemos demo end-to-end: la cajera arma la OP, el supervisor la aprueba, consolida en OC.

Lo que falta para cerrar el ciclo:

1. **Registrar la compra** cuando llegue la mercadería del proveedor.
2. **Notificarle al ERP** de Diego que entró stock para que actualice su sistema.

> **Diseño elegido**
>
> Las OP/OC *nuestras* viven en Supabase (no las guardamos en SQL Server porque son entidades nuevas que el ERP no necesita conocer). Pero **cuando se registra una Compra**, ahí sí necesitamos avisarle al ERP para que actualice el stock central. Es el único punto de "escritura" que necesitamos del lado del SP.

2) `MULTICOMPRA_COMPRA_REGISTRAR`

```
@Reporte = 'MULTICOMPRA_COMPRA_REGISTRAR'
```

¿Para qué sirve?

Cuando llega la mercadería al depósito central, la supervisora abre la OC en MultiCompra, tilda lo que llegó vs lo pedido, y confirma. En ese momento la PWA llama a este reporte para que **se registre la entrada de stock** en el sistema de Diego. La factura del proveedor también se carga acá.

¿Qué le manda la PWA?

A diferencia de los reportes anteriores, este recibe muchos parámetros + un JSON con las líneas:

Parámetro	Tipo SQL	Descripción
`@Reporte`	`varchar(50)`	Siempre `'MULTICOMPRA_COMPRA_REGISTRAR'`
`@OCNumero`	`varchar(50)`	El número de la OC nuestra (ej: `OC-loc-K3M5N`)
`@IDProveedor`	`int`	El ID del proveedor en tu sistema
`@IDDeposito`	`int`	Depósito donde entra la mercadería (normalmente el central)
`@FacturaNumero`	`varchar(50)`	Número de factura del proveedor
`@FechaRecepcion`	`datetime`	Fecha de recepción (default: GETDATE())
`@UsuarioRecibe`	`int`	ID del usuario que verifica la mercadería
`@Lineas`	`nvarchar(max)`	JSON array con `[{ "IDItem":1, "CantidadRecibida":10, "CostoUnitario":7000 }, ...]`
`@Observaciones`	`nvarchar(500)`	Texto libre (opcional)

¿Qué tiene que hacer el SP?

1. Validar que el proveedor exista y esté activo. Si no, devolver error.
2. Insertar registro en la tabla de `Compra` de tu sistema (lo que aplique en ADCON2).
3. Para cada línea del JSON, insertar en `DetalleCompra` y aumentar el stock en el depósito indicado.
4. Actualizar el costo del producto si cambió (con la lógica de "ActualizarCosto" que ya tenés).
5. Devolver el ID interno de la Compra para que MultiCompra lo guarde.

¿Qué tiene que devolver?

Columna	Tipo	Ejemplo	Notas
`ok`	`bit`	`1`	1 si todo OK, 0 si falló
`idCompra`	`int`	`15847`	El ID de la compra creada en tu sistema (para auditar)
`numeroCompra`	`varchar`	`C-001-001-12345`	Número que tu sistema le asigna
`totalGs`	`decimal`	`1250000`	Total calculado
`productosActualizados`	`int`	`18`	Cantidad de productos cuyo stock se actualizó
`mensaje`	`varchar(200)`	`Compra registrada y stock actualizado`	Texto descriptivo

Boceto T-SQL

```
If @Reporte = 'MULTICOMPRA_COMPRA_REGISTRAR' Begin

    Declare @vIdCompra      int = 0
    Declare @vNumeroCompra  varchar(50) = ''
    Declare @vTotalGs       decimal(14,2) = 0
    Declare @vCantProd      int = 0
    Declare @vOk            bit = 0
    Declare @vMensaje       varchar(200) = ''

    Begin Try
        Begin Transaction

        -- 1) Validar proveedor
        If Not Exists (Select 1 From Proveedor Where IDProveedor = @IDProveedor And Activo = 1)
            Set @vMensaje = 'Proveedor inválido o inactivo'
            GoTo CompraCerrar
        End

        -- 2) Insertar Compra
        Insert Into Compra (IDProveedor, FechaCompra, FacturaNumero,
                            IDUsuarioRecibe, Observaciones, IDDeposito)
        Values (@IDProveedor, IsNull(@FechaRecepcion, GetDate()), @FacturaNumero,
                @UsuarioRecibe, @Observaciones, @IDDeposito)

        Set @vIdCompra = Scope_Identity()
        Set @vNumeroCompra = 'C-' + Right('00000' + Cast(@vIdCompra As varchar), 5)

        -- 3) Insertar líneas y actualizar stock
        Declare @lineas table (
            IDItem int, CantidadRecibida int, CostoUnitario decimal(14,2)
        )
        Insert Into @lineas
        Select IDItem, CantidadRecibida, CostoUnitario
        From OpenJson(@Lineas)
        With (
            IDItem int '$.IDItem',
            CantidadRecibida int '$.CantidadRecibida',
            CostoUnitario decimal(14,2) '$.CostoUnitario'
        )

        -- Detalle
        Insert Into DetalleCompra (IDTransaccion, IDItem, Cantidad, CostoUnitario)
        Select @vIdCompra, IDItem, CantidadRecibida, CostoUnitario From @lineas

        -- Stock (asumiendo tabla StockDeposito o similar)
        -- Aca viene la logica que vos sabes mejor: actualizar stock por
        -- (IDItem, IDDeposito) sumando CantidadRecibida.

        -- Actualizar costo del producto si corresponde
        Update I
        Set Costo = L.CostoUnitario
        From Item I Inner Join @lineas L On I.IDItem = L.IDItem
```

```
        Where I.ActualizarCosto = 1

        Set @vTotalGs = (Select Sum(CantidadRecibida * CostoUnitario) From @lineas)
        Set @vCantProd = (Select Count(*) From @lineas)
        Set @vOk = 1
        Set @vMensaje = 'Compra registrada y stock actualizado'

        Commit Transaction
    End Try
    Begin Catch
        If @@TranCount > 0 Rollback Transaction
        Set @vMensaje = 'Error: ' + Error_Message()
    End Catch

    CompraCerrar:
    Select 'ok' = @vOk, 'idCompra' = @vIdCompra,
           'numeroCompra' = @vNumeroCompra, 'totalGs' = @vTotalGs,
           'productosActualizados' = @vCantProd, 'mensaje' = @vMensaje

End
```

Preguntas

1. ¿Cómo se llama exactamente la tabla de Compra y la de DetalleCompra en ADCON2?
2. ¿Cómo se actualiza el stock por (IDItem, IDDeposito)? Si es vía un SP existente, podemos llamarlo desde acá.
3. ¿Querés que también dispare automáticamente el `MULTICOMPRA_RECALCULO_STOCK` al final, o lo manejamos del lado PWA?

3) `MULTICOMPRA_OP_GUARDAR` (opcional, baja prioridad)

```
@Reporte = 'MULTICOMPRA_OP_GUARDAR'
```

¿Para qué serviría?

Hoy las OPs se guardan SOLO en MultiCompra (Supabase). Si por algún motivo querés que también queden registradas en el ERP de Diego (para reportes históricos cruzados con ventas), este endpoint las copia.

> **Decidí si vale la pena**
> Si las OPs son "intención de pedido" y solo importa el resultado final (la Compra registrada), **no hace falta esto**. Si querés trazabilidad cruzada con tu sistema, sí. Mi recomendación: dejar las OPs en Supabase y solo loggear la Compra final en el ERP.

Spec si decidimos hacerlo

Parámetro	Tipo	Descripción
`@Reporte`	`varchar(50)`	`'MULTICOMPRA_OP_GUARDAR'`
`@OPNumero`	`varchar(50)`	El ID legible de la OP (ej: `OP-loc-X9Y2Z`)
`@IDDeposito`	`int`	Depósito que pidió
`@IDUsuarioCajera`	`int`	Cajera que la armó
`@FechaEnvio`	`datetime`	Cuándo se envió a la supervisora
`@Estado`	`varchar(20)`	`'enviada'` | `'aprobada'` | `'cerrada'` | etc.
`@Lineas`	`nvarchar(max)`	JSON con `[{ "IDItem", "Cantidad" }]`

Devolvería: `ok`, `idOPLog`, `mensaje`.

Implementación: insertar en una tabla nueva `OPHistorial` (que crearías en ADCON2) con el espejo de la OP. Solo para auditar.

4) Próximos pasos

1. Vos: implementás `MULTICOMPRA_COMPRA_REGISTRAR` (lo crítico). Pasame las preguntas que te queden y armamos juntos.
2. Yo: armo el cliente Supabase para guardar OPs y OCs (ya está stub listo).
3. Validamos en demo con Diego que el flujo cierra completo: cajera pide → supervisora aprueba → consolida en OC → llega mercadería → se registra y el stock del ERP se actualiza.

4. Si Diego pide auditoría cruzada, hacemos también `MULTICOMPRA_OP_GUARDAR`.

Documento generado el 28 de abril de 2026 · MultiCompra Fase 2 · NODO · `github.com/MULTIPLAZA/multicompra`